

CS3305: Team 14 Report

MediMate

By Mark Vaughan (123446326), Sean Dunlea (122353906), Tracy Lazarus (123388563) and Amina Baig (123482946)

[GitHub Repository](#)

Academic Integrity Declaration

We declare that this report and the project work are our own original work. All sources used in the development of this project have been acknowledged.

Table of Contents

1. [Introduction](#)
 - [Problem & Competitors](#)
 - [Our Solution](#)
2. [Workflow](#)
3. [System Architecture](#)
 - [High-Level System Architecture](#)
 - [System Data Flow](#)
 - [Database Schema](#)
4. [Features](#)
 - [User Authentication & Management](#)
 - [Dashboard & User Analytics](#)
 - [Medication Management](#)
 - [MediMate Management with Privacy Controls](#)
 - [Symptom Logging](#)
 - [Notification System](#)
 - [Background Scheduler](#)
 - [AI-Powered Summaries](#)
 - [User Interface & Experience](#)
5. [Roles & Responsibilities](#)
 - [Mark Vaughan](#)
 - [Sean Dunlea](#)
 - [Tracy Lazarus](#)
 - [Amina Baig](#)
6. [Table of Contributions](#)
7. [Challenges & Lessons Learned](#)
8. [Areas for Improvement](#)
9. [Conclusion](#)
10. [Use of Generative AI](#)
11. [References](#)

1. Introduction

Problem & Competitors

Have you ever worried about whether you or someone close to you has taken their medication on time? Medication adherence is a very common challenge in healthcare, especially for the elderly, people with illnesses, or simply those who just tend to forget. Missing medications or taking it at the wrong time can negatively impact one's overall health.

While a number of solutions already exist in helping this matter, many medication trackers simply focus on basic reminder functionality. Applications such as **MyTherapy**, **MediSafe**, and **CareClinic** allow users to set reminders and track medication schedules, but they fail to include strong social accountability features or deeper insights into a user's behaviour over time. Additionally, many of these are designed primarily for mobile applications, with very limited web-based alternatives.

As a result, users struggle with maintaining a good habit of keeping track of their medications. This also creates a sense of anxiety for those who care for elderly parents, relatives, or loved ones, as they may worry about whether medication routines are actually being followed correctly.

Our Solution

To combat this issue, we present MediMate - a medication tracker web application which is designed to help users manage their medications more effectively.

A web-based platform was chosen not only because there are limited web-based solutions available, but also because it allows the app to be accessed from any device with a browser, without having to install additional software. Since our target audience includes elderly users, this approach makes it slightly more accessible for both patients and their caretakers.

The application allows users to schedule their medications, log their taken doses, and monitor adherence through a personalised dashboard. The dashboard includes a weekly colour-coded status, a progress chart displaying adherence over time, statistics for streak, total adherence percentage, and total active medications. We also provide a weekly AI generated summary of their medication activity. This allows users to keep on top of their routine and better understand their habits.

The key feature of our system is the 'MediMates' functionality - A feature allowing users to share their progress with trusted friends, families, or caretakers. These MediMates can view the user's medication activity, and send reminders if a dose has not been logged. The aim of this feature is to create a sense of support for the user, and motivating them to stay consistent. The system also includes automated reminders through push notification and email alerts to help users remember scheduled doses. If the user misses a dose after the scheduled time, an alert will be sent to the MediMate, allowing them to provide encouragement or reminders.

Our app goes a step further by providing users a space to log the symptoms they've experienced. They are given the option to print or download a compilation of their symptoms as a report, along with a report of their current medications to share with their doctor.

The overall goal of our app is to help the user manage their medication well and provide reassurance for the people responsible for their care.

2. Workflow

After our group had been created, we set up a meeting as early as possible to get to know each other and begin brainstorming ideas for what we could build for our software project. We came up with a few ideas before we voted on MediMate to be our project.

We discussed how we could build upon a medicine tracker app before deciding that a helpful feature that had not been worked upon would be to split the responsibility of remembering to take your medication among others. We talked about how we could implement this with a notification system.

Once we had a solid idea of how to go about creating the project, we divided the work between us. We made the decision to split up the work through each persons personal preference and their skills.

Before starting the development process, we created a shared google docs to write down all the tasks we had to complete.

Each week we would have a mandatory meeting in person, and additional online meetings to ensure work was getting done and to bring up problems team members might have been facing in order to solve them faster and not have one team member stuck on a bug for too long.

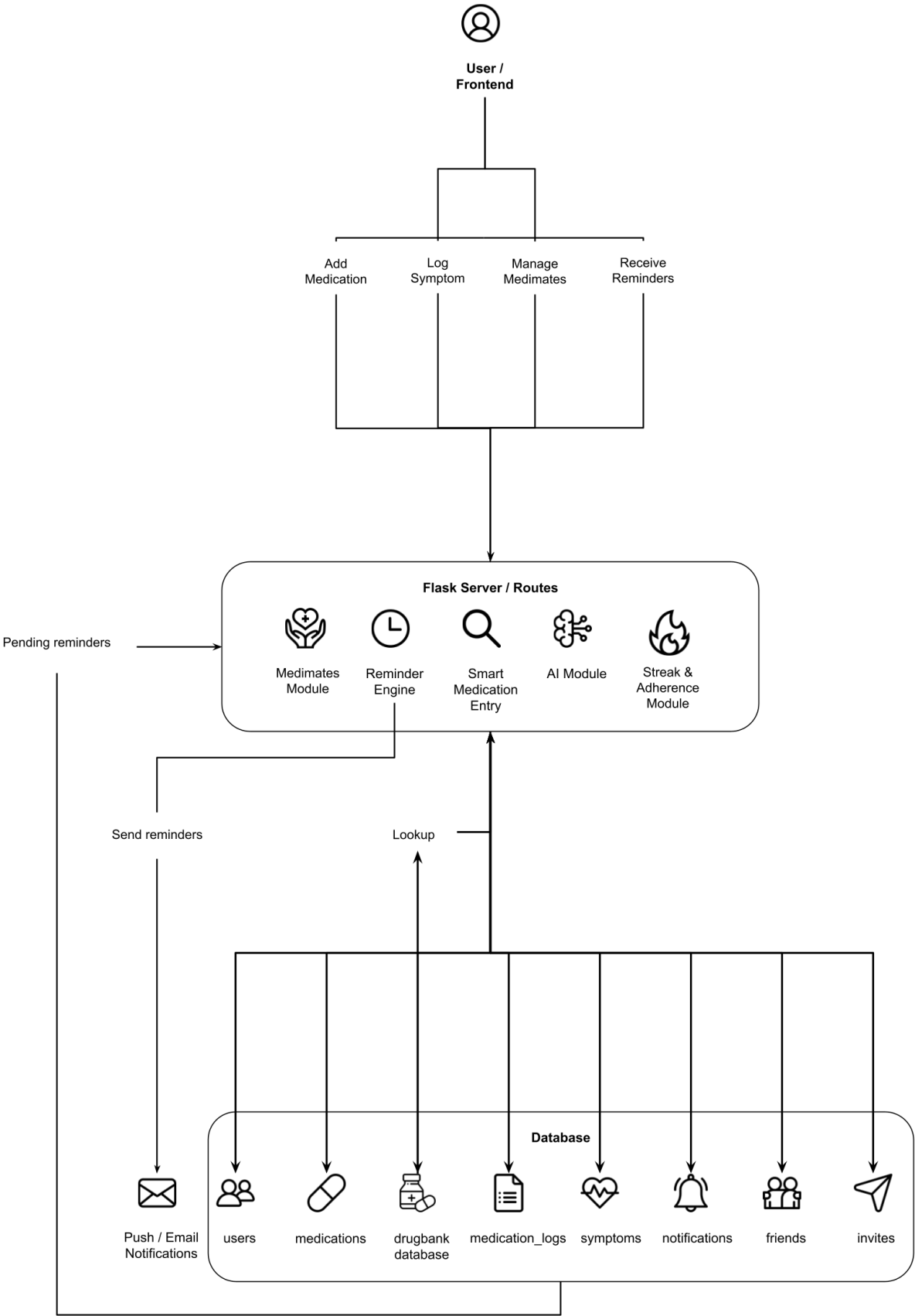
For the time between where the team would have meetings we created a WhatsApp group chat as a way to ask and answer quick questions we had on others' work or to make requests of other members that had arose.

Toward the end of the development process the code was tested with different ways we thought may cause an error in order for us to find bugs and fix them before the project had to be presented.

To prepare for the presentation we had a meeting to see what were the most important parts to present. We each took parts that related to our work and decided the best order to put the slides in to make it as linear as we could. We practiced the presentation multiple times in order to get more familiar with our parts and to deliver a presentation that flowed well.

3. System Architecture

High-Level System Architecture



As outlined in the High-Level System Architecture Diagram, our MediMate system uses a layered web application architecture, consisting of a user interface, a backend application server and a persistent

database.

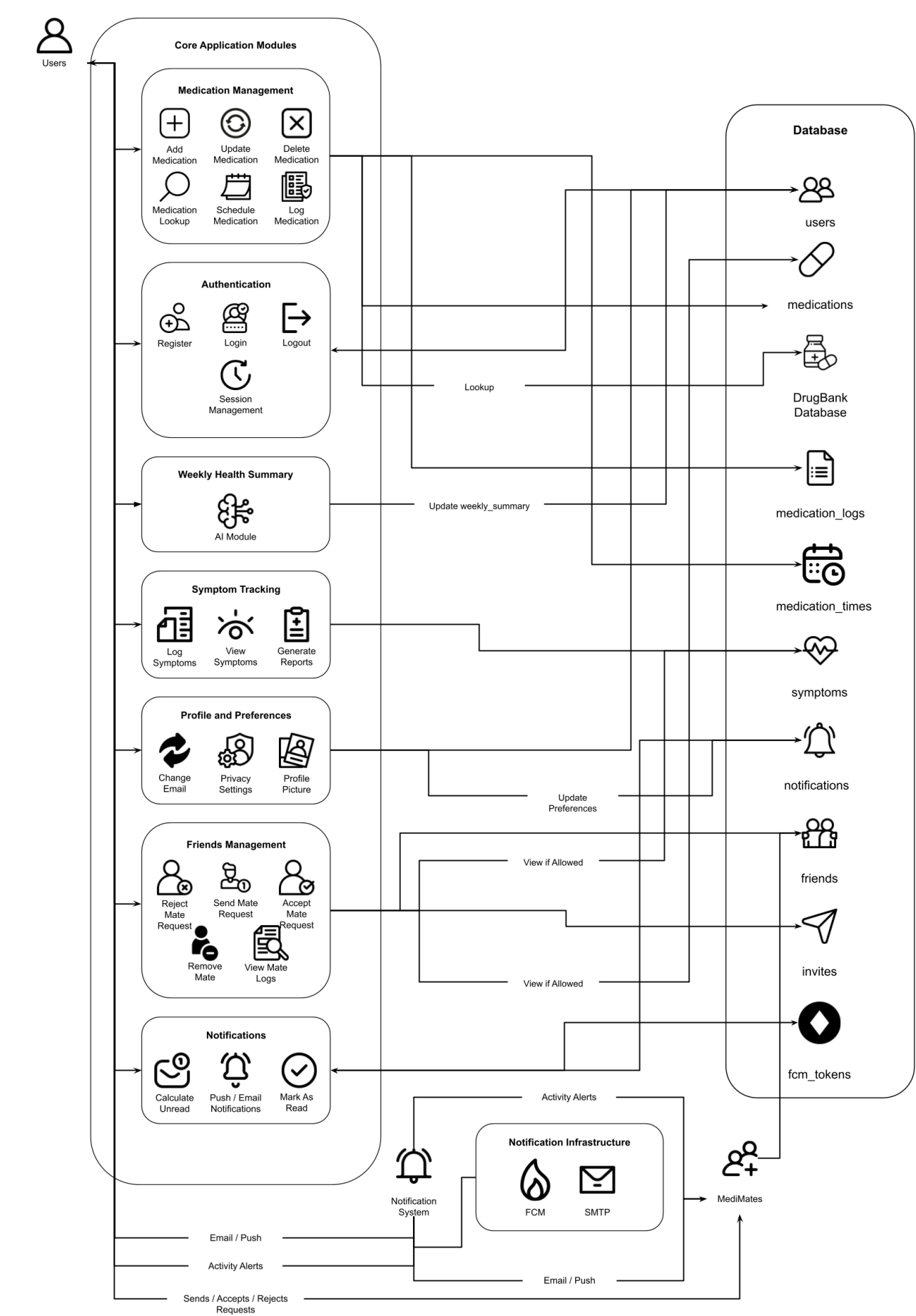
The frontend provides an intuitive interface through which users can perform core actions such as adding medication, logging symptoms, managing MediMates and receiving reminders. User requests are transmitted to the backend via **HTTP**.

The backend processes user actions through a set of specialised modules, namely, the MediMates module which manages social connections between users, the reminder engine which schedules medication reminders to be delivered through email or push notifications, the smart medication entry system which facilitates the efficient lookup of medications, the AI module which analyses user symptoms and medication intake to produce health summaries, and the streak calculator which tracks users' adherence streaks.

The database layer stores all of our system's persistent data, including user information, medications, symptom logs, notifications, friends, invites and a local DrugBank database that supports autocomplete functionality. Each backend component interacts with the database, as required, ensuring all modules operate on consistent and up to date information.

This layered approach enhances modularity, resulting in a more maintainable and scalable system.

System Data Flow



This diagram illustrates the primary data flows between users, the backend application modules, the database layer and the notification infrastructure within our MediMate system.

The medication management component processes user requests related to medications. When adding, deleting or scheduling medications, entries in `medications`, `medication_logs` and `medication_times` are updated accordingly. Additionally, a preprocessed subsection of the DrugBank dataset, stored locally, enables users to search for medications quickly.

The authentication component controls user authentication and session management, supporting user registration, login, logout, as well as session tracking. Interactions within this module are limited primarily to the users table to validate credentials and maintain user sessions.

AI-powered summaries provide user health reports, utilising aggregated data from `medication_logs` and `symptoms`. These insights are stored in `users` to be presented subsequently.

The symptoms component allows users to log and view health observations. Records are stored in `symptoms`, and later retrieved for analytical purposes.

Users may update account information through the profile and preferences component, which provides centralised control over email updates, privacy configurations and additional user data. These changes are reflected in the database to ensure user preferences are respected across the system.

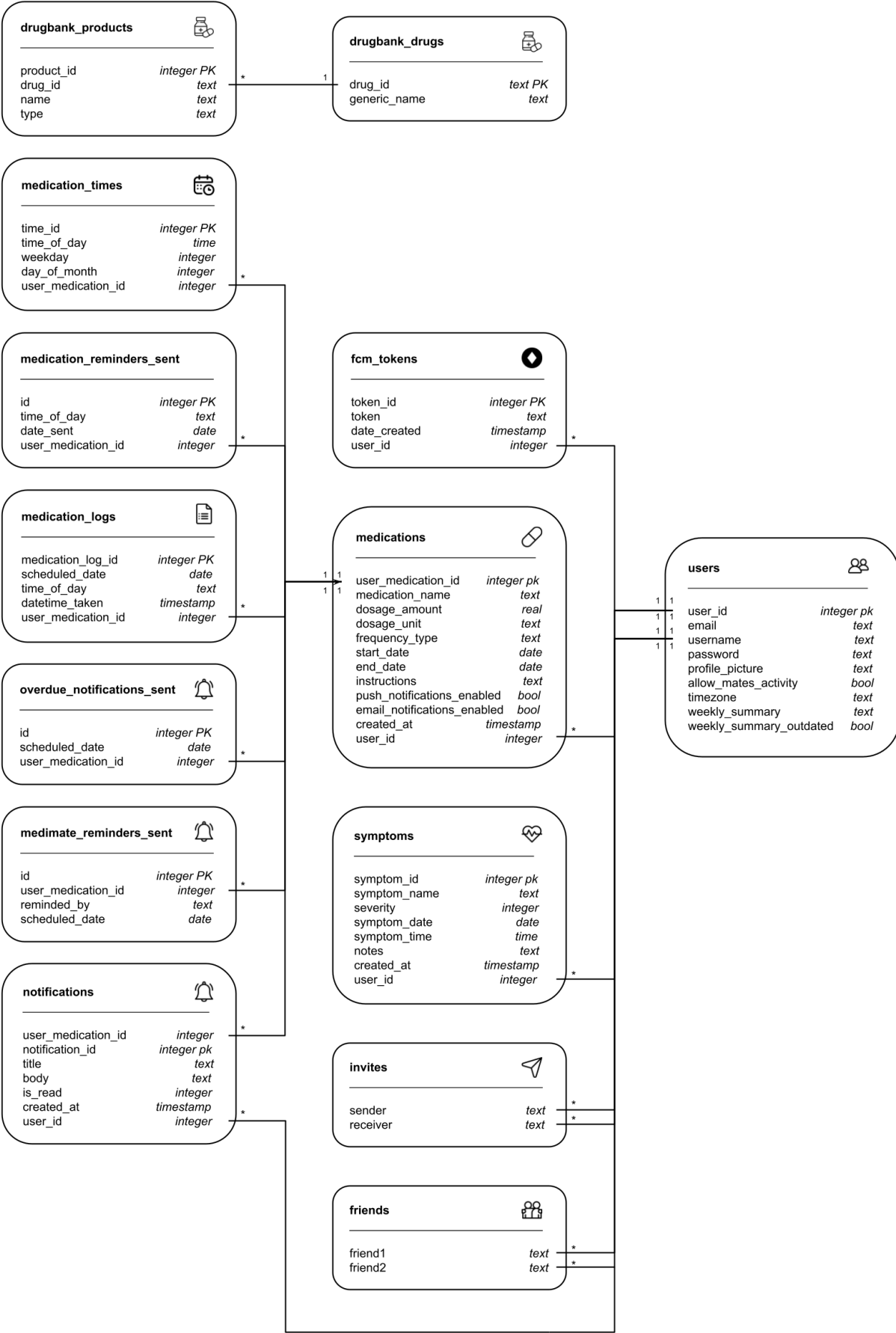
The MediMates component captures social connections between accounts. Users can send, accept or reject mate requests, as well as remove existing connections. Relationships are represented in `friends` and `invites`, determining whether users can view each other's medication activity and symptom logs.

Notifications are handled through a dedicated module, which calculates unread messages and manages user-facing alerts. Triggered at scheduled medication times, or perhaps due to MediMate actions, the system generates appropriate alerts to be delivered through our notification infrastructure.

The notification infrastructure utilises external services to deliver reliable messages to users. Firebase Cloud Message (FCM) delivers push notifications, while Gmail's SMTP server handles email notifications. Stored FCM tokens and notification preferences ensure dependable and consistent delivery.

Each backend component interacts with the database layer, which functions as the central repository for our application's data. This ensures all features operate on the latest data.

Database Schema



As demonstrated in the Database Schema Diagram, our application utilises a relational database to store all persistent data.

`users` stores the primary account information, including username, email, password, profile picture and additional user preferences. `allow_mates_activity` provides data privacy autonomy, allowing users to limit the visibility of personal information from medimates. `medication`, `symptoms`, `notifications` and `fcm_tokens` reference `users` through foreign keys, making it a primary driver for all user-related data in our system.

`medications` reserves information of medications registered by users. Each entry contains the medication name, dosage information, frequency and duration. A single entry belongs to a specific user and serves as the central reference point for related data, including reminder schedules, medication logs and notifications.

`medication_times` defines the medication intake schedule for each user. Entries specify timestamps and recurrence patterns for medications, allowing our system to support various scheduling configurations (e.g daily/weekly/monthly).

When a user confirms medication intake, the event is stored in `medication_logs`. Each entry contains the scheduled date, the time of confirmation and the corresponding medication reference. These records provide the historical data required for adherence tracking, streak calculations and weekly health summaries.

`medication_reminders_sent` and `overdue_notifications_sent` record sent notifications, preventing duplicate alerts by ensuring that reminders are delivered only once per scheduled event.

`medimate_reminders_sent` stores reminders generated by the MediMates component, when friends prompt each other to stay consistent with their medication routines. This data is then used to implement safeguards, ensuring users are not spammed with messages.

All notifications are stored in `notifications`, for display in our in-app notification centre. Each entry records the message title, content, read status and timestamp.

Our system supports push notifications through Firebase Cloud Messaging (FCM). FCM tokens are required to send push notifications, which are stored in `fcm_tokens`. Each token is associated with a specific user, allowing the notification infrastructure to deliver alerts directly to their registered devices.

Symptom tracking functionality is implemented through `symptoms`. Each entry documents user-reported symptoms, including severity, time of occurrence and optional notes. These records are then analysed for inclusion in our AI-powered health summaries.

To support social accountability features, our system includes tables for managing relations between users. `invites` records pending friend requests, while `friends` stores established connections between users. These relationships aid in determining the visibility of user activity within our application.

Finally, a preprocessed local copy of the DrugBank dataset supports efficient medication lookup. `drugbank_drugs` and `drugbank_products` store medication names and associated product information, allowing the system to provide fast autocomplete search suggestions as users add new medications.

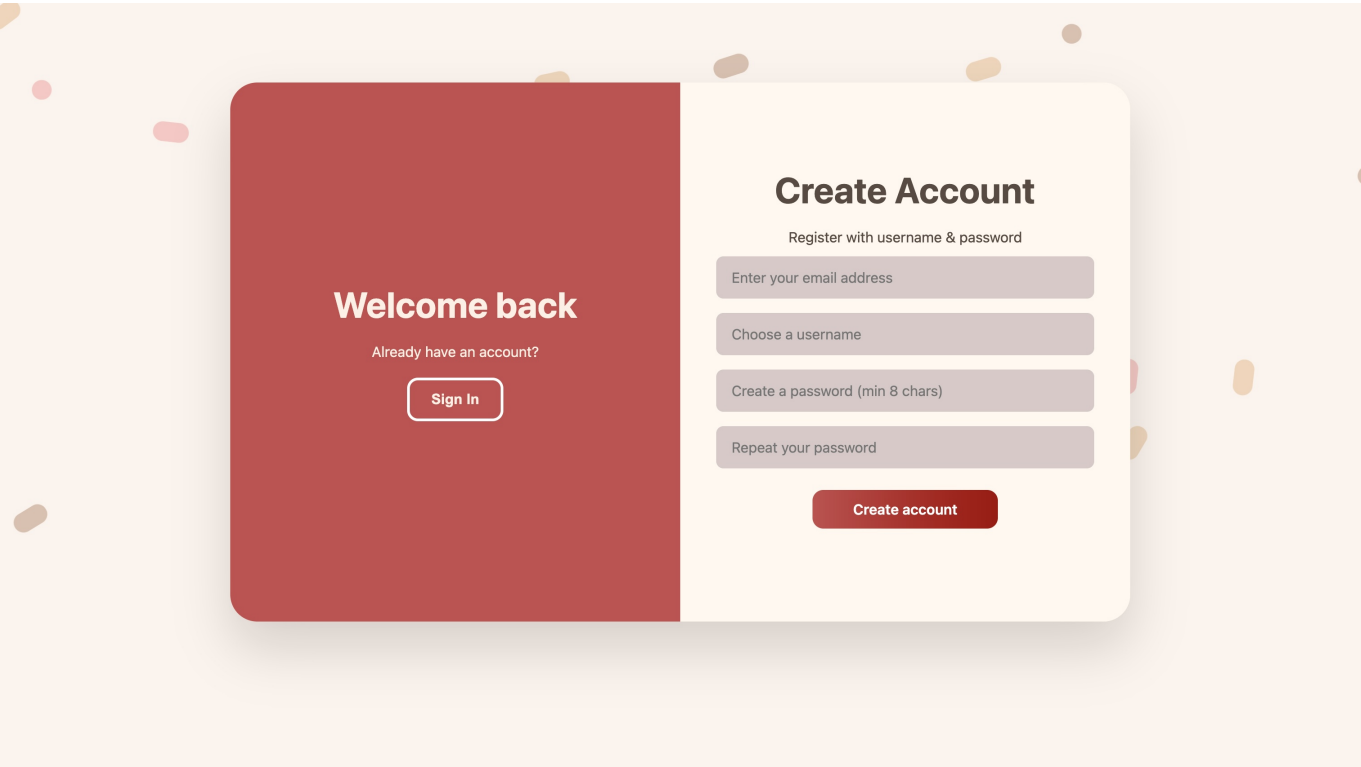
These tables form a relational schema that supports the core functionality of MediMate, including medication scheduling, adherence tracking, symptom logging, social accountability features and notification delivery. Through the separation of responsibilities across specialised tables while linking them through foreign key relationships, our schema achieves flexibility and consistency for system evolution.

4. Features

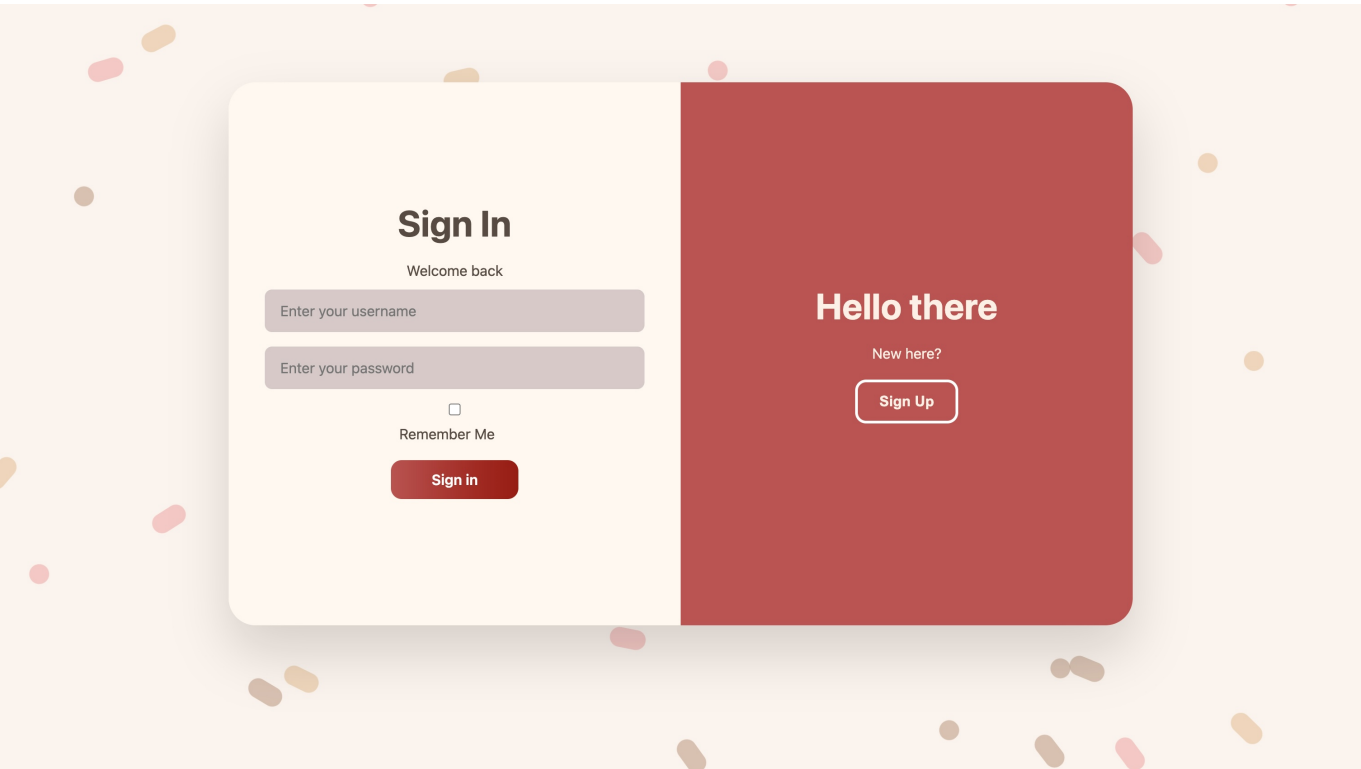
User Authentication & Management

Our system provides secure user registration and login with username or email and password. Emails are authenticated before registration, ensuring only valid addresses are accepted. The application maintains user sessions through a "Remember Me" option, allowing users to stay logged in across several browser sessions.

Registration Interface



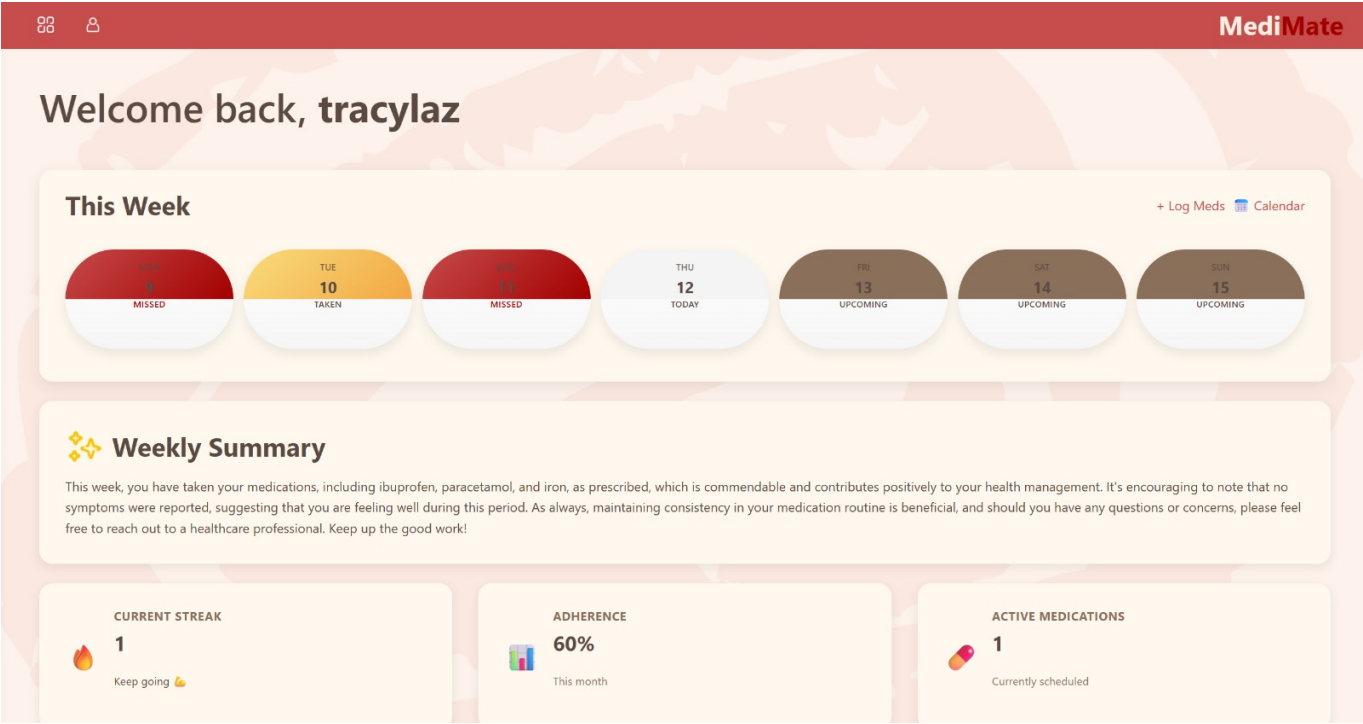
Login Interface



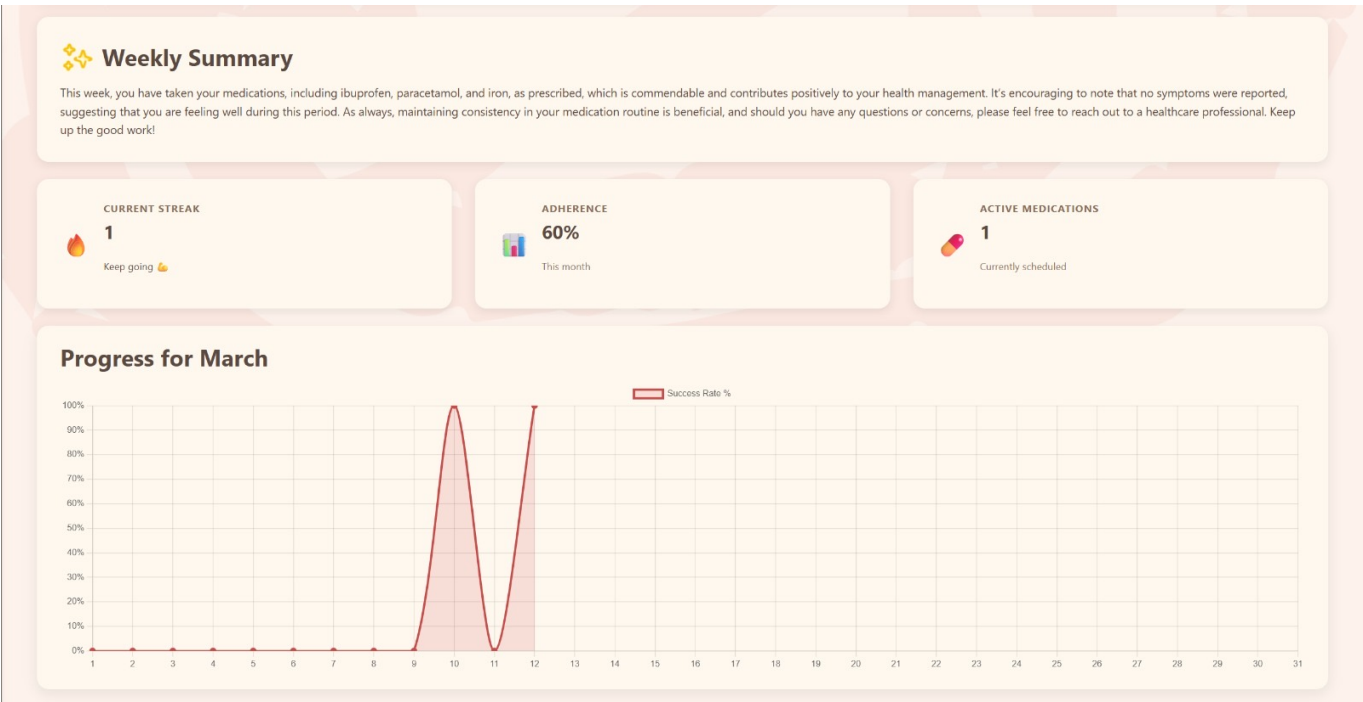
Dashboard & User Analytics

The dashboard provides a central overview of recent user activity, with key insights into their medication routine. Users can quickly view upcoming medication events and adherence analytics, including streak tracking for consecutive days where medications were taken as scheduled. These streaks aim to encourage consistency and reinforce positive self-care habits. By visualising adherence patterns and recent activity through the dashboard, users can quickly assess how well they are maintaining their medication schedule.

Dashboard & User Analytics Interface (1/2)



Dashboard & User Analytics Interface (2/2)



Medication Management

Users can add and log medications with customisable schedules, including daily, weekly, monthly or as-needed reminders. Our system supports specifying dosage, timing and duration for each medication, as well as dynamic field adjustments based on the selected frequency type. A smart medication entry system, powered by the DrugBank database, provides autocomplete suggestions with generic names, brand names and synonyms for reliable searching. Validation logic prevents erroneous entries. The calendar provides an intuitive interface through which users can view scheduled medications in upcoming months. Users may also view past and current medications, with the ability to print or save this information for quick reference.

Add Medication Form (1/4)

Medi

Add Medication

Medication Details

Medication Name

Ibupr

Ibuprofen

Ibuproxam

Ibuprofene

Ibuprofeno

Dosage

10

mg

Add Medication Form (2/4)

Medi

Add Medication

Frequency

Times per Period

1

Daily

Time of Day

13:40

+ Add Another Time

Back

Next

Add Medication Form (3/4)

Medi

Add Medication

Schedule

Start Date

12/03/2026

End Date

19/03/2026

Further Instructions (Optional)

e.g. Take with food

Back

Next

Add Medication Form (4/4)

Medi

Add Medication

Notifications

Push Notifications?

Email Notifications?

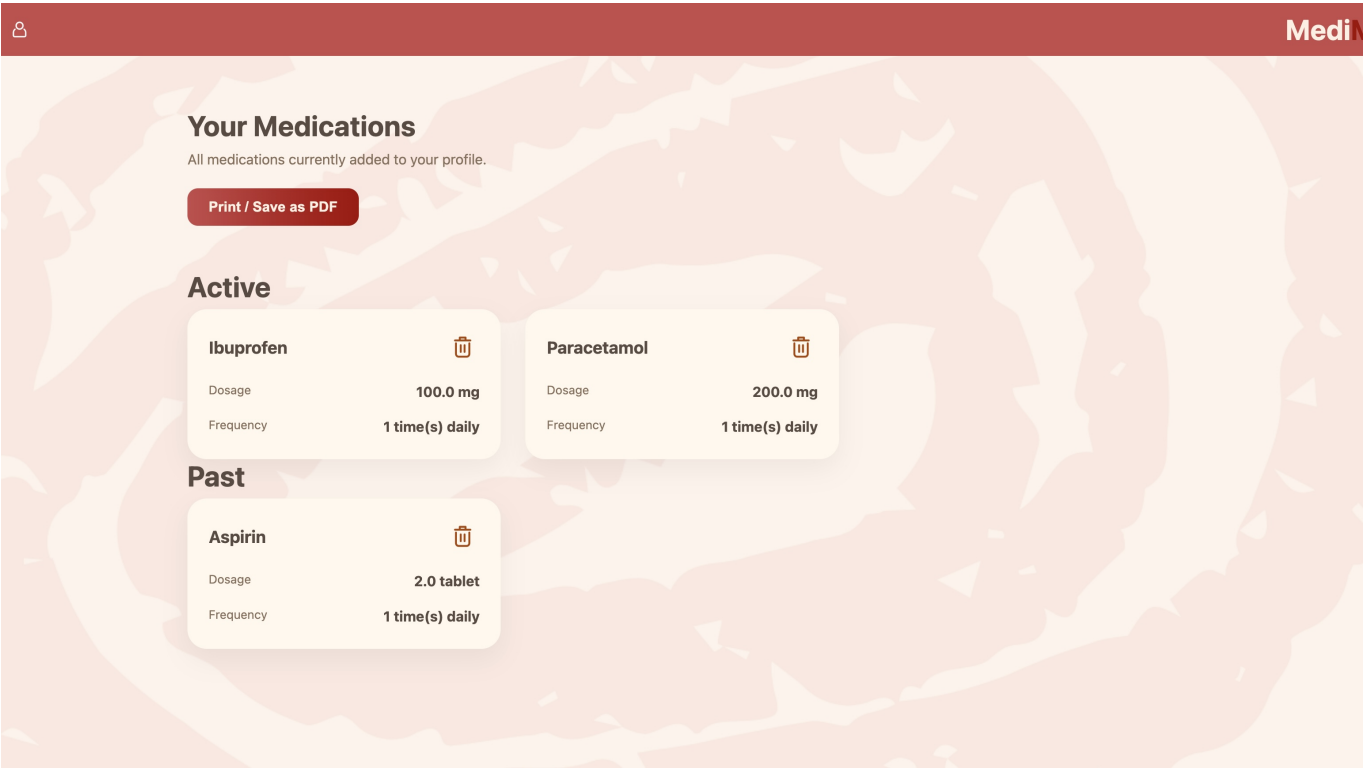
Back

Submit

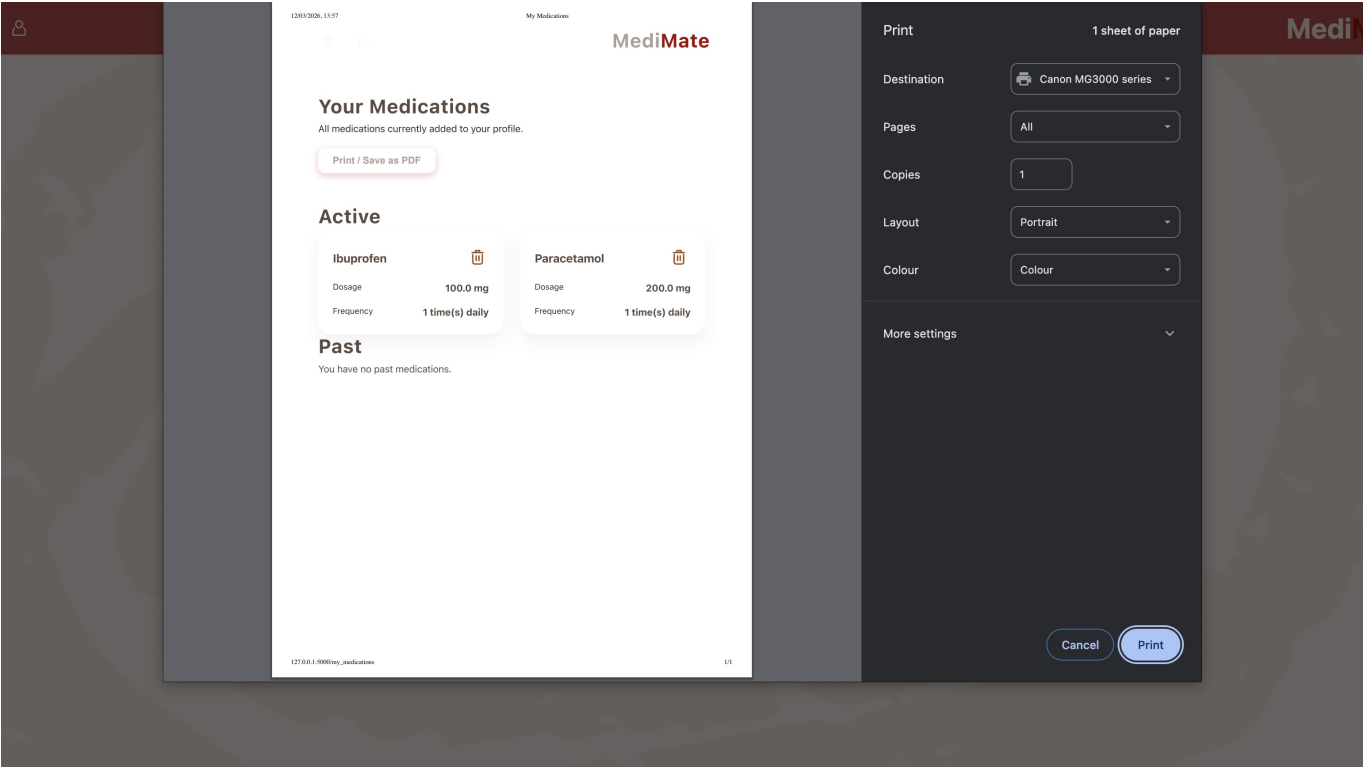
Calendar Interface



Medication Management Interface (1/2)



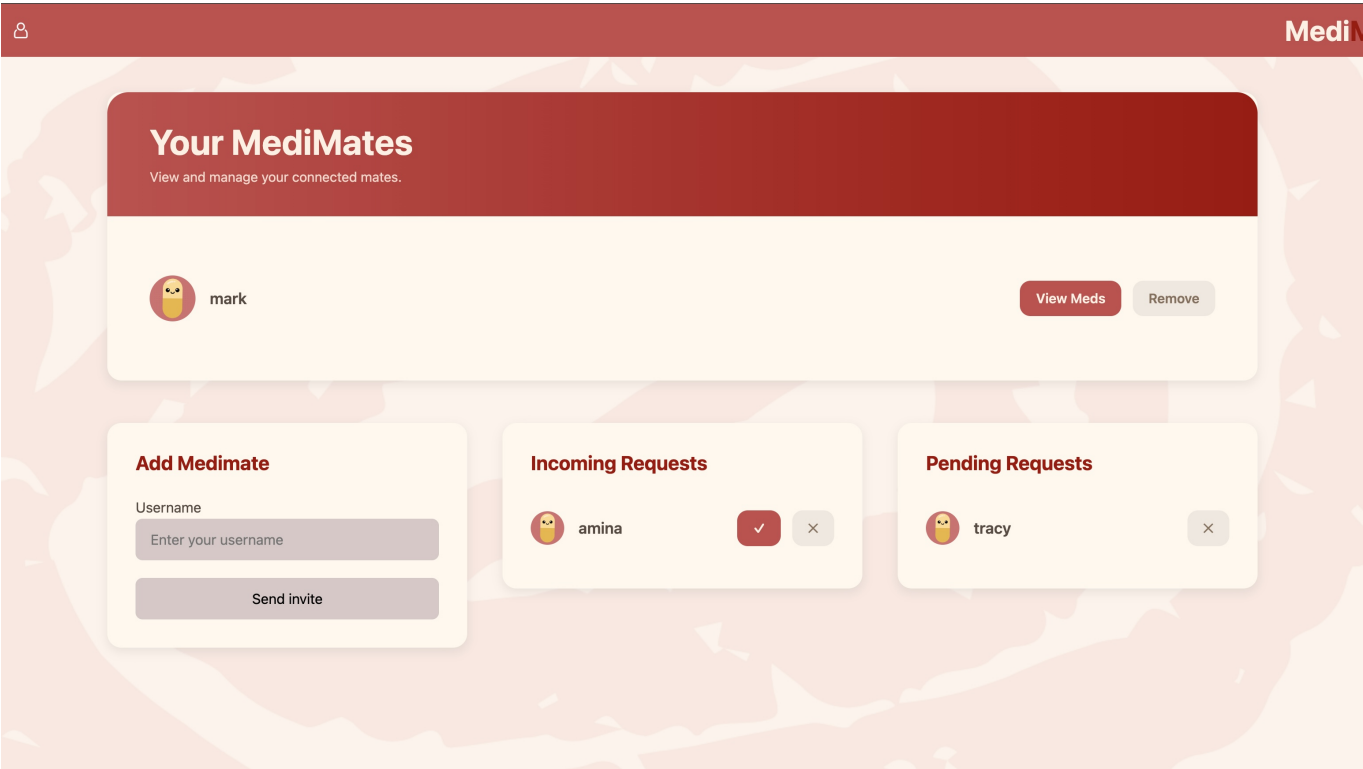
Medication Management Interface (2/2)



MediMate Management with Privacy Controls

This is the key feature that distinguishes our application from similar medication trackers. Users may add other users as MediMates to provide support and accountability. If user settings permit, MediMates may view each other's medications, logged symptoms and receive notification if a dosage is missed, allowing for gentle re-reminders. Users can manage their MediMates list, adding or removing friends as needed to maintain a supportive network.

MediMates Interface



MediMate's Activity Interface

Medi

mark's Medications

Ibuprofen

Dosage: 100.0 mg

2026-03-12 → 2026-03-19

mark's Logged Symptoms

Fever

Severity: 3

2026-03-12

Privacy Control Settings

Change Password

Delete Account

Privacy

Allow your mates to view your activity☒

Save Preferences

Symptom Logging

Users can log symptoms they experience alongside their medications. The symptom logging system allows users to track patterns overtime and provides input for generating personalised AI health summaries. Logged symptoms are stored securely and can be viewed and managed through the user dashboard.

Symptom Log Interface

MediMates

Log a Symptom

Record how you're feeling today.

Symptom details

Symptom

Fever

Severity

3 - Moderate

Date

12/03/2026

Time

13:00

Symptom History

Share Report

No symptoms logged yet.

Notification System

Our system implements multiple notification channels, ensuring users never miss their medications. Browser push notifications are delivered via Firebase Cloud Messaging, and email reminders are sent through the integrated email system. Users can also view all notifications through our in-application notification centre, and if a medication is missed by one hour, their designated MediMates are notified to provide a gentle reminder.

Friend Request Notification



medimate.alerts@gmail.com

New Mate Request! 

billy_I has sent you a MediMate request!



New Mate Request! 

127.0.0.1:5000

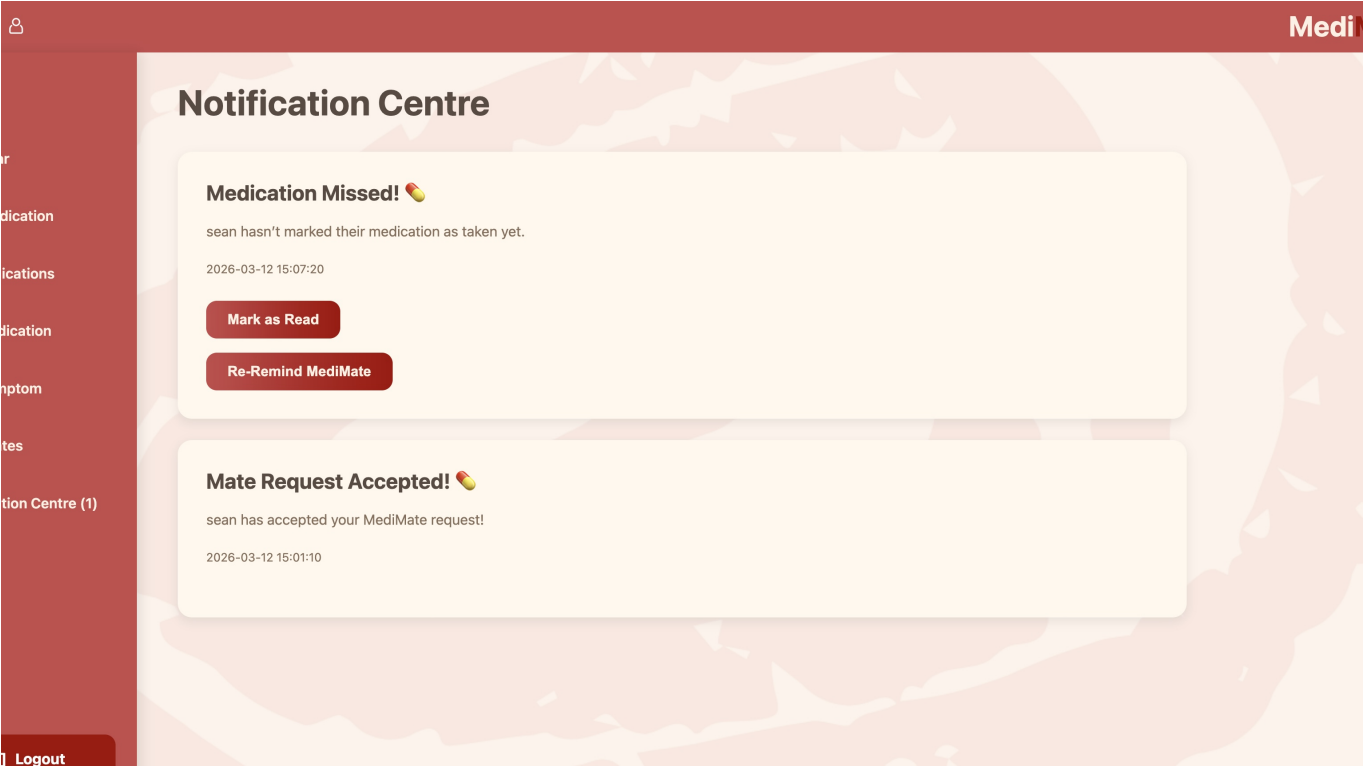
billy_I has sent you a MediMate request!



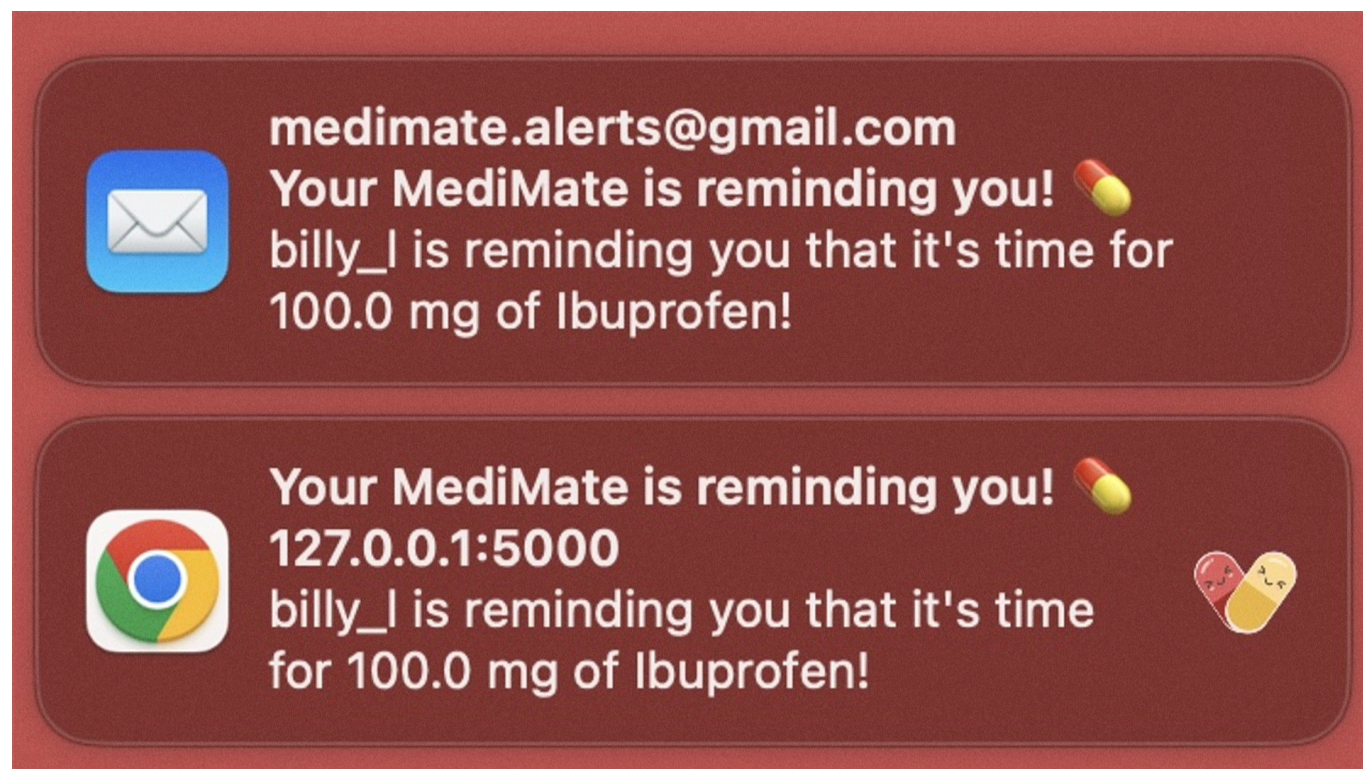
Medication Reminder



Notification Centre Interface



Overdue Alert



Background Scheduler

The background scheduler triggers reminders at the precise time for each user using timezone-aware scheduling. It continuously evaluates medication schedules with minute-level precision to compute upcoming doses, detects overdue medications and triggers MediMate escalation, as required. The system also includes safeguards to prevent duplicate notifications and ensure reliable delivery.

AI-Powered Summaries

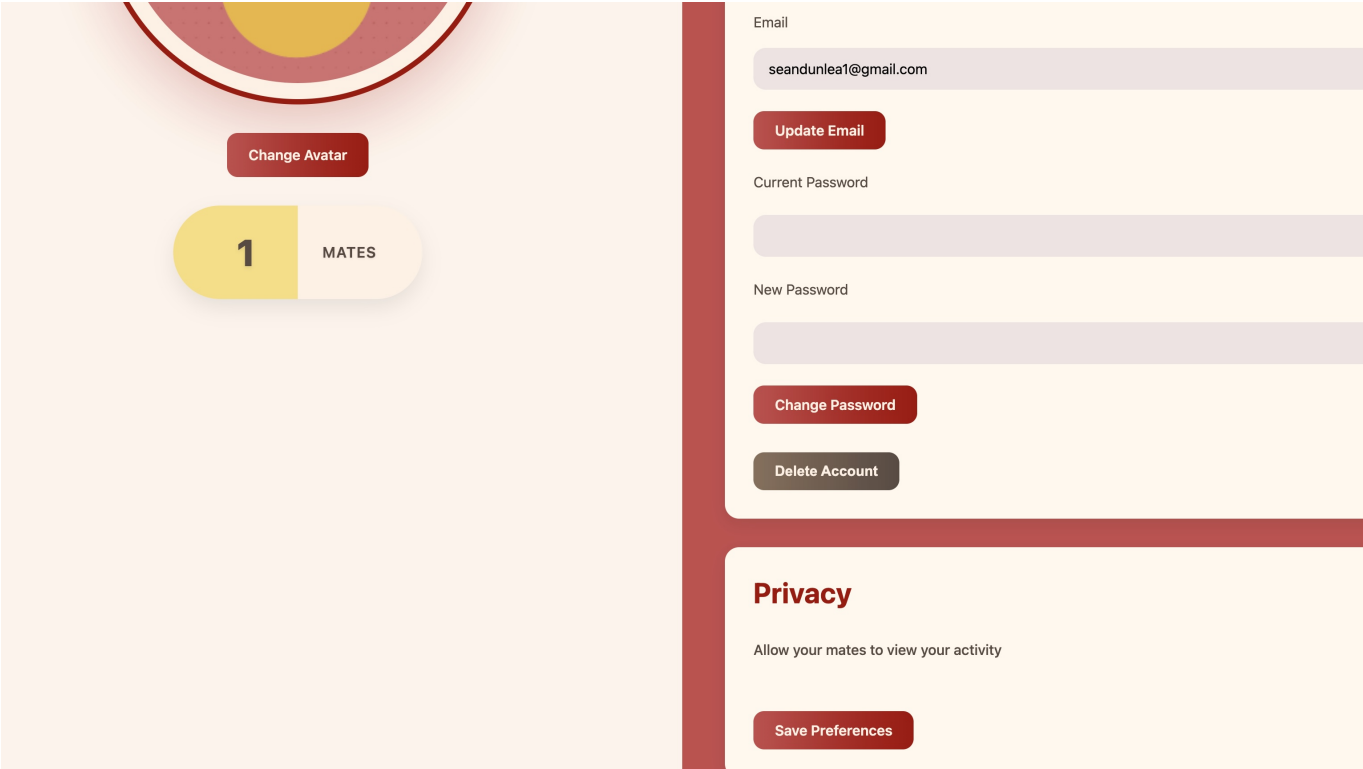
Our system generates personalised health summaries based on logged medications and symptoms. The AI provides friendly and encouraging messages for users who follow their medication schedule and gentle reminders for missed doses. Data privacy is ensured, as no personally identifiable information is included in the summaries.

Weekly Summary

This week, it appears that while some medications were taken as scheduled, aspirin was not logged, which is an important part of your regimen. You did take paracetamol and ibuprofen, which could be beneficial for managing symptoms. You experienced a mild fever, and it's good to be mindful of how you feel during this time. If your fever persists or worsens, or if you have any concerns about your medication routine, it may be helpful to consult with your healthcare provider for further guidance. Remember, maintaining open communication about your health is key, and you are taking positive steps by keeping track of your symptoms and medications.

The system provides users with a dedicated profile and setting section where they can manage personal information. Users can change the email or password associated with their account, enhancing user agency. They can also opt-in for push notification reminders, or control the visibility of their activity to MediMates.

User Settings Interface (2/2)



User Interface & Experience

The application offers a responsive interface with intuitive navigation across pages. Notifications and reminders are supported across multiple devices, enhancing accessibility and convenience.

5. Roles & Responsibilities

Mark Vaughan (123446326) - Backend Developer and Debugger

My primary focus was to work on the backend logic of the app and specifically on the friend system called "**MediMates**". This was done through the use the use of the following technologies: **HTML, Jinja, Python (Flask) and SQL**.

The main section I worked on was the MediMates section. This section of the app handles all the features to do with the friend system that the app uses. This required a form to take the name of a user, check if they exist and send them a friend request. Friend relationships would be stored in a table with a reversed duplicate entry making it easy to gather friend data for use without gathering duplicates of relationships. Two sub-sections were made for the MediMate section, one showing the pending requests that a user had sent to other users and the other being incoming requests from other users.

I worked on the privacy feature for users accounts. I added a column in the user schema that stored if users allowed for the privacy settings in the profile page. I then worked across existing parts of the app to implement checks to enforce privacy if the user has chosen to share information with MediMates or not. I made modifications to the notification functions in order to make them adhere to the privacy settings. If they allow for info to be shared their MediMates will receive reminders if they have not taken their medication after an hour. Otherwise the schedule does not send any notification to the users MediMates.

Another feature optional feature dictated by the choice of privacy settings chosen was if friends could see your saved medications and symptoms logged. If information sharing is allowed in settings, if your friend

goes into their MediMates page they will see a new link that would otherwise not appear, allowing them to see details regarding your recent symptoms you have logged such as the severity and notes and also information on your medications. This was done by making queries to both the `medications` table and the `symptoms` table through the flask route which would take the users `user_id` to join all these tables together to access the data. The feature allows for concerned friends or family to have a way to check any symptoms the user may have logged but forgot to inform them about providing a useful history.

I merged pages together that were originally separate. I took the originally separate profile and settings page and merged them into a single flask route that feels more connected and simple. The same was done with the friends page which originally gave all the features on the page their own route and HTML page. These were all put together to create a more complete friends page making it so the user would have to spend less time looking for the links to carry out different actions to do with their MediMates.

I spent a lot of time testing features across the entire app searching for any possible bugs that may be present in the code. I did this by testing many various entries in forms across the app to make sure that they worked as intended and then tested other features they worked with features that used the inputted data e.g. how entered medication worked with the calendar features. I found the calendar originally set all future medication as missed so I added new status words and conditional statements in order to prevent users from being confused when viewing the month and week calendars.

Overall my role focused on developing the friends system of the page which serves as a backbone to the idea and uniqueness of our app and searching for bugs across the entire app that may cause problems for users.

Sean Dunlea - Backend and Systems Developer

As a backend and systems developer, my primary responsibility was building the backend server architecture. This involved implementing several advanced features for our application such as the authentication and user management system, medication recording and scheduling functionality, the push and email notification infrastructure, an in-application notification centre, the background reminder scheduler, a smart medication autocomplete system and AI-powered health summaries. These features required careful co-ordination of multiple technologies, including Flask, SQLite, JavaScript, Firebase Cloud Messaging (FCM), as well as external APIs.

One of the first components I developed was the authentication and user management infrastructure. I implemented secure registration and login functionality using Flask, WTForms and session-based authentication. Passwords are hashed using `Werkzeug`, and I took further security measures by providing generic login error messages to mitigate user enumeration attacks. The registration process includes validated email addresses, ensuring authentication sessions are managed reliably across the application.

Next, I worked on building our notification infrastructure. After researching several options, I decided to integrate FCM to support browser push notifications, as relying solely on email notifications may result in missed reminders. Implementing this system required developing both frontend and backend components. On the frontend, I wrote JavaScript to request user notification permissions, register service workers, retrieve FCM tokens and synchronise those tokens with the backend server. On the backend, I developed routes and helper functions to send notifications to all devices associated with a specific user, while automatically removing invalid or expired tokens. This ensures notifications remain reliable, even when users access the application from multiple different browsers or devices. In addition to push notifications, I implemented an email notification system by creating a dedicated Gmail account and integrating it with

Flask-Mailman and **Gmail SMTP**. This ensures that users still receive medication reminders even if push notifications are unavailable or disabled. I also developed an in-application notification centre, allowing users to view their notifications directly within the user interface.

After this, I worked on designing and implementing the database structure and backend logic needed for reliable medication management. This required developing an efficient schema that supports user-specific medications, configurable medication schedules and multiple intake times per day. I created functionality that allows users to add medications with various scheduling patterns, including daily, weekly, monthly and as needed frequencies. I also wrote JavaScript to support dynamic form input, allowing users to specify dosage, duration and multiple daily intake times. To ensure data consistency, I implemented validation logic that ensures only valid medication schedules may be submitted.

I implemented a background scheduling system using **APScheduler** to support medication reminders. This is a Python library designed for scheduling code execution at specified times or intervals. The scheduler runs periodically and checks medication schedules at minute-level precision, determining whether reminders should be sent or not. One challenge I faced during this process was ensuring that reminders get delivered at the correct time for users in different time zones. To address this, I implemented JavaScript that detects the user's timezone in the browser and transmits this data to the backend. Here, it is stored in **users** and retrieved for calculating reminder times. The scheduler converts the user's local timezone before triggering notifications. I also designed database mechanisms to prevent duplicate reminders and implemented logic to detect overdue medications. If a medication is not recorded as taken within one hour of the scheduled time, the system automatically notifies the user's designated MediMates and allows them to send additional reminder notifications.

To enhance user experience, I developed a smart medication entry system, allowing users to search for and find medications efficiently. Initially, after doing some research, I explored the RxNorm dataset. However, I found its structure unsuitable for the intended purpose as it separated medications into extremely granular variations (e.g. distinguishing between base drug names and specific dosage forms) and relied on US-centric naming conventions. I researched alternative resources and contacted DrugBank to request API access. DrugBank is a "trusted intelligence operating system built on the most comprehensive biomedical knowledge base". Although the API was not available for academic use, I was granted access to their full database dataset. As the dataset was very large (approximately 2GB), I implemented a preprocessing pipeline to extract only the relevant medication name information required for autocomplete functionality. I then created a separate database schema and integration logic so that the dataset could be automatically initialised without other team members having to manually manage the large source file. This allows our application to provide intuitive autocomplete suggestions while keeping the repository manageable.

Finally, I implemented the AI-generated health summary feature. I explored several possible solutions before choosing OpenRouter as the most suitable API choice for our project. Some of my earlier approaches using OpenAI and Hugging Face came with limitations in cost and storage size to store large local models. Using OpenRouter allows our application to generate summaries dynamically without requiring large local resources. I designed prompts that summarise medication adherence and symptom logs in a supportive tone. The summaries encourage users when medications are taken as scheduled, gently highlight missed doses and provide general feedback on recorded symptoms, while avoiding any diagnostic language. To protect user privacy, the data provided to this API is anonymised, and contains no personal identifiable information such as usernames or email addresses. To improve performance and minimise API calls, I implemented caching logic that stores the generated summary and only regenerates it when relevant user data changes, for example, when a new symptom is logged or if a new medication event occurs.

Overall, my role focused on backend development, notification infrastructure, system architecture and the integration of advanced features that support the functionality of our MediMate application. Through my work, I gained a lot of experience designing and integrating full stack systems, especially in coordinating backend logic, client-side functionality and third party services in a reliable, maintainable and scalable manner.

Tracy Lazarus - Full-Stack Feature Developer

My primary responsibility was to create features that users would interact with across both the frontend and backend. This involved displaying the features on the interface using HTML and Jinja templates, and doing the backend logic with Python (Flask). I integrated database operations to complete features of the system. In total, the features I worked on required technologies such as Flask, HTML, Jinja templates, JavaScript, and SQL. As I worked on both frontend and backend, my role required me to work closely with both the UI Developer and Backend Developers.

The first feature I worked on was improving the authentication that was already built. I developed a 'Remember Me' feature that optionally allows users to remain signed in. This required linking the frontend form inputs with the backend session management so that the app will correctly store and retrieve the authentication state. I also built a Logout route which allows the user to end the session.

Next, I made a symptom logging page. I developed a form that allows users to record symptoms they experienced which includes input fields like symptom name, severity, date, time, and optional notes. When submitted, these symptoms will display in a section called Symptom History. For this, I had to create a symptoms table in the schema. These symptoms are displayed in chronological order. To make this feature more useful, I added in a print option which allows the user to download or print a compilation of their symptoms as a PDF report that can be shared with their doctor.

The main feature I worked on was the dashboard. It provides the user with visual insights of their medication progress. Here, I implemented a seven-day calendar view which shows whether doses were taken, missed, or not scheduled yet. This was the most challenging as designing this required calculating the current week and mapping each medication schedule to the current day. It had to also account for different medication frequency patterns which required additional logic. In the dashboard, I also implemented an adherence streak counter, a monthly adherence progress line chart which I had to use JavaScript for, dashboard statistics which includes the user's current adherence percentage and the number of active medications. This provides better insight into medication habits.

To allow the users to interact with the system, I made a Log Medications page where users can mark medications as taken. They can only do this for previous and current days but not for the future. The logging system then updates the calendar view in the dashboard automatically.

In addition to the weekly tracking system, I created a full monthly view of the calendar which serves the same purpose of showing logged medication status but over longer periods. I put in a slider here so that the user can go back and forth between months to view the history of the full year.

Finally I worked on the profile and settings section of the app. The section allows users to view their account information such as the number of MediMates they have, and they can select a profile avatar. In settings, I added a functionality to allow users to change their email address and password, as well as update their privacy settings that allow users to pick whether they want their mates to see their activity or not.

Overall, my role focused on making the core user-facing features and ensuring the frontend, backend, and database worked together effectively.

Amina Baig - Frontend & UI Developer

As the Frontend and UI Developer, my role involved designing the complete visual interface of MediMate. This involved using CSS, JavaScript and rewriting all existing HTML and Jinja templates to ensure they worked with the styling and layout of the application. Occasionally, I also updated app routed in app.py to work with any frontend features, such as colour-coding calendars. During our initial discussions on how to separate the work, I decided to volunteer to focus mainly on the visual design of the system as this allowed the interface to remain visually consistent, while allowing the other team members to dedicate time to developing the backend features. My work focused on improving overall usability, consistency, and creating the visual structure of the app so that the features could be easily understood by users.

At the beginning of the project, I created an initial prototype design to help visualise the layout and navigation of the application before development began. This allowed the team to understand how the main pages such as the dashboard, medication management pages and authentication screens would be structured. I mainly created hand-drawn wireframes and then improved those digitally using Canva.

I selected the theme colours of the app to be red and yellow. These colours were chosen because according to the research paper 'The Influence of Colour on Memory Performance' by Mariam Adawiah Dzulkifli and Muhammad Faiz Mustafar (referenced below), warm colours such as red and yellow have been shown to attract attention and support memory retention, which is relevant to MediMate where users are trying to keep track of their doses.

After the prototype stage, I implemented the interface using CSS, JavaScript and rewriting existing templates. Since our application targets users who may include elderly patients and sick people, usability and readability were important considerations. I designed the interface to be simple and clear, avoiding overly complex layouts. For multi-step processes like adding medication, I made the UI go one step at a time for ease of use.

I was responsible for styling all pages in the application. To ensure consistency across the application, I defined global CSS variables for color/size and some reusable styling patterns so that elements such as buttons, navigation links, and form inputs followed the same design language. I also used JavaScript for to do some animations to make the interface more interactive such as for a sidebar and add medication steps.

I worked closely with the team to ensure that newly implemented features could be properly displayed in the user interface. This involved adapting layouts when new data structures were introduced and ensuring that forms correctly submitted data to the backend routes.

To keep other aspects of our project consistent, I also designed our presentation and video pitch.

Overall, my role focused on transforming the backend functionality into a cohesive, user-friendly and visually consistent interface. By designing the layout, implementing the styling system and ensuring that templates integrated properly with Flask and Jinja, I helped ensure that MediMate remained easy to navigate and accessible for its users.

6. Table of Contributions

Name	Contribution
Mark	MediMates: - Created the MediMates HTML page - Created the schema for the friends database to allow for the join of two users through a duplicate reversed relationship entry for each friendship - Created schema to store MediMate invites holding the sender and receiver - Made a form through WTForms to handle MediMate invites. - Created a route to reject MediMate requests and another to accept and add them to the friends database while removing the entry from the invites database - Created a flask route to cancel sent MediMate requests and delete them from the invites database - Created a route to remove a MediMate and remove the relationship and its reversed version from the friends database - Implemented logic to make mutual friend requests automatically add the friends to the database
	Privacy Settings - Created the logic for the privacy form - Adjusted logic in the notifications section to only send MediMates reminders if you allow for it in privacy settings - If the user allowed for their information to be shared I created a route that would be accessible through the MediMates page to show friends your medications and symptoms logged
	Medications - Split the My Medications page to display current medications being taken and past medications
	Merging pages - Joined the originally split profile page and settings page into one, more connected page - Merged all MediMate based features into one page to make actions relating to adding and removing friends more simple
	Testing and Fixes - Tested features across the app in order to try seek out bugs and correct them - Added additional checks and statuses to add to the calendar to prevent future days and the current day for showing as missed or having no status
Sean	Project Setup and Infrastructure: - Created the initial Flask project structure and repository - Set up the application environment and dependency management - Helped manage the shared GitHub repository and resolve integration issues during development Authentication and User Management - Implemented secure user registration and login using Flask, WTForms and session

Name	Contribution
	authentication - Implemented password hashing using Werkzeug - Added email functionality and validation - Implemented security improvements such as generic login error messages to prevent user enumeration Database Design and Backend Architecture: - Designed database schemas for users, medications, medication schedules, notification tracking and related tables - Implemented backend logic for medication management and user-specific medication schedules - Developed support for multiple medication intake times and configurable schedule frequencies (daily, weekly, monthly and as-needed) Medication Autocomplete System: - Researched medication datasets, including RxNorm and DrugBank - Integrated a medication autocomplete system to improve user experience when adding medications - Parsed and preprocessed the DrugBank dataset to extract relevant medication names - Designed a database schema and initialisation logic so the dataset could be used without requiring manual setup by teammates Notification Infrastructure: - Researched and implemented push notifications using Firebase Cloud Messaging (FCM) - Developed client-side JavaScript to request notification permissions, register service workers and synchronise device tokens - Implemented backend logic to send push notifications and manage multiple tokens per user device - Added automatic cleanup of invalid or expired tokens Reminder Scheduling System: - Implemented a background medication reminder scheduler using <code>APScheduler</code> - Designed scheduling logic that checks medication schedules every minute and triggers reminders with minute-level precision - Implemented timezone detection and conversion to ensure reminders occur at the correct local time for each user - Built logic to detect overdue medications and notify MediMates when a medication is missed Email Notification System: - Implemented email notification functionality using <code>Flask-Mailman</code> and <code>Gmail SMTP</code> - Created a dedicated email account for the application - Integrated email reminders alongside push notifications to ensure reliable reminder delivery Notification Centre: - Implemented a notification centre where users can view and manage their notifications

Name	Contribution
	- Implemented logic to store and retrieve notifications from the database
	AI Integration: - Implemented AI-generated health summaries using the OpenRouter API - Designed prompts that summarise medication adherence and symptom logs in a supportive manner - Ensured anonymised data was sent to the AI service to protect user privacy - Implemented caching logic to reduce unnecessary API calls and improve application performance
	Debugging and Integration: - Debugged backend, database and frontend integration issues throughout development - Ensured features remained stable when integrating new components

Name	Contribution
	Authentication: - Implemented a Logout route - Created a 'Remember Me' option allowing users to stay signed in Symptom Logging System: - Made a form allowing users to log the symptoms they experienced - Created a Symptom History section in the same page that displays all added symptoms - Designed database schema for symptoms - Implemented a Share Report option which allows the user to download or print the symptoms history as PDF Medication Management: - Added a history page of added medications called 'My Medications' which retrieves and displays added medications - Created a Delete Medication option in My Medications which removes the added medication from the database
Tracy	Medication Tracking Dashboard: - Developed a colour-coded 7-day calendar view for scheduled medication to show the days where medications that were taken, not taken, and not scheduled - Added a functionality to mark doses as taken and store logs in the database - Implemented a streak counter for consecutive days of successful medication intake - Developed monthly adherence progress chart, using JavaScript - Added dashboard statistics which includes current adherence percentage and total active medications Calendar: - Created a full view monthly calendar which shows the status of whether medications were taken or not each day - Added a slider to navigate between all of the months in the year Profile and Settings - Implemented a profile section showing account information such as number of mates and the ability to choose a profile picture - Included an option to change email and password - Added privacy settings which controls whether mates can view your activity

Name	Contribution
Amina	User Interface Design & Prototyping: - Designed the overall visual layout and navigation structure of the application - Created initial hand-drawn wireframes to plan the structure of key pages such as the dashboard, authentication pages and medication management interfaces - Digitised design concepts using Canva to share with the team and refine the interface layout Frontend Development: - Implemented the visual interface using CSS and JavaScript - Rewrote and restructured HTML and Jinja templates so that backend data could be displayed correctly within the styled interface - Ensured consistent styling and layout across all pages of the application Styling System & Layout: - Defined global CSS variables for colours, spacing and sizing to maintain a consistent design system - Created reusable styling patterns for interface elements such as buttons, forms, navigation links and page sections - Designed layouts that prioritised readability and simplicity for users Interactive UI Features: - Implemented interactive elements using JavaScript to improve usability - Added animations and dynamic UI behaviour such as sidebar interactions and step-based medication form transitions - Assisted with frontend logic required for visual features such as colour-coded calendar displays Graphic Assets & Visual Elements: - Designed and integrated graphic elements used throughout the application - Created avatar graphics used within user profiles - Designed and implemented the application favicon - Implemented animated visual elements on the homepage to improve engagement and visual appeal Frontend–Backend Integration: - Worked with backend developers to ensure new features were properly displayed in the user interface - Updated templates and occasionally adjusted Flask routes to support frontend functionality - Ensured forms correctly submitted user input to backend routes Project Presentation Design: - Designed the visual layout of the team presentation and video pitch materials to maintain a consistent visual identity

7. Challenges & Lessons Learned

Version Control

The biggest problem we faced during the development of the project was using Git. As multiple team members were working on the same repository at the same time, merge conflicts occurred occasionally when changes were made to the same sections of code. At the beginning, it was tough as we all did not have much experience using Git in a collaborative space. Sometimes, when trying to solve a merge conflict, code would end up being removed accidentally.

However, as time went on, we learned to review the code carefully to ensure no functionality was lost and we made sure to communicate to each other in the group chat or directly to the person who was in charge of that specific part of the code about what had happened.

Allocating Work

As the project started progressing and the number of features increased, it sometimes became unclear which tasks should be prioritised and who should be responsible for completing them. After we completed the main development tasks, there were still some smaller features and fixes. Without a clear allocation, there was a risk that some tasks would've been forgotten or multiple people would have worked on the same task unnecessarily.

To fix this problem, we used a shared task list on Google Docs containing all the remaining project tasks. We organised this checklist inspired by the **MoSCoW** method, where we categorised each task based on their importance. The tasks highlighted in green represented the must-do tasks that were essential for the application and the ones in orange represented the should-do or could-do tasks that could be completed if we had time left. We had a column beside each task where our team members could assign their names. This helped prevent working on the same code simultaneously.

Time Management

Another challenge we faced was arranging times for the team to meet. Although we met every Wednesday, we needed additional discussions to make decisions about the project. However, as team members had different schedules and academic commitments (in our case working on another team project), finding suitable times was slightly difficult.

To overcome this issue, we decided to schedule meetings online in advance a few times if we were unavailable to meet in person.

Frontend Integration Challenges

One challenge during development was making sure the frontend templates worked properly with dynamic data from the Flask backend. Our pages rely on Jinja templates to display information such as medications, notifications and dashboard statistics. Due to this, when backend routes or database queries changed, the HTML templates often needed to be adjusted so the loops, conditionals and variables would render correctly without breaking the layout or styling.

We communicated in our groupchat along with reading the commit messages to know when the templates needed updating.

Working through this helped us better understand how Flask passes data to templates and how Jinja is used to dynamically generate the user interface.

8. Areas for Improvement

Although MediMate successfully demonstrates the core features we thought were required, we had several more ideas that could have improved the app if we had more time.

One important area is accessibility. Since the application is designed to support elderly users, additional features for accessibility such as a voice-over support, larger text mode or a dark/light theme toggle could have assisted users who may have visual impairments or difficulty navigating interfaces.

Another potential idea we had would be the addition of an in-built messaging feature between MediMates. This would allow caretakers or trusted mates to communicate directly within the application and send reminders or encouragement when the user's medications have not been logged. These messages could also trigger optional email notifications.

9. Conclusion

From a technical perspective, this project shows how multiple components can come together to make a cohesive system. The application combines a Flask-based backend, a relational database, a frontend interface, background scheduling for reminders, and services such as Firebase Cloud Messaging and AI APIs.

Beyond the technical aspect, the project provided us as a team, valuable experience in software development. We learned how to communicate effectively, manage version control, coordinate work, and organise tasks effectively.

Overall, MediMate shows how a software solution can help support healthier habits.

10. Use of Generative AI

Generative AI tools such as ChatGPT were used during the development process for debugging purposes, when all team members did not know the issue. All final code, comments, and report were written and reviewed by the team.

11. References

Competitors

MyTherapy <https://www.mytherapyapp.com/>

MediSafe <https://www.medisafe.com/>

CareClinic <https://careclinic.io/>

Research

The Influence of Colour on Memory Performance (Mariam Adawiah Dzulkifli, Muhammad Faiz Mustafar) <https://pmc.ncbi.nlm.nih.gov/articles/PMC3743993/>